

Chapter 8 — Software Testing

TRUE / FALSE — SLIDE-BY-SLIDE EXAM NOTES

Every line from every slide → converted to exam-ready True/False statements

● GREEN = TRUE | ● RED = FALSE | 💡 = Exam Tip

📌 HOW TO USE THESE NOTES:

Cover the T/F column → read each statement → say your answer aloud → uncover to check. Focus extra on RED (FALSE) statements — examiners love to flip exact slide wording to trick you!

SLIDE 2 — Topics Covered

Statement (from Slide)	T / F
The four main topics covered in Chapter 8 are: Development testing, Test-driven development, Release testing, and User testing.	TRUE
Chapter 8 covers five main topics including integration testing. <i>☞ Only 4 topics: development testing, TDD, release testing, user testing.</i>	FALSE
Test-driven development is one of the topics covered in Chapter 8.	TRUE
Release testing is NOT covered in Chapter 8. <i>☞ Release testing IS one of the four main topics.</i>	FALSE

SLIDE 3 — Program Testing

Statement (from Slide)	T / F
Testing is intended to show that a program does what it is intended to do.	TRUE
Testing is intended to discover program defects before the program is put into use.	TRUE
Testing is intended to discover program defects AFTER the program is put into use. <i>☞ Defects must be found BEFORE the program is put into use.</i>	FALSE
When you test software, you execute a program using artificial data.	TRUE
When you test software, you execute a program using real production data. <i>☞ Testing always uses ARTIFICIAL data, not real production data.</i>	FALSE
You check the results of the test run for errors, anomalies or information about the program's non-functional attributes.	TRUE
Testing can reveal the presence of errors but NOT their absence.	TRUE
Testing can prove that a program is completely error-free. <i>☞ Testing reveals PRESENCE of errors, not ABSENCE — it can never prove a program is error-free.</i>	FALSE
Testing can reveal the absence of errors in a program. <i>☞ Testing can NEVER prove absence of errors.</i>	FALSE
Testing is part of a more general verification and validation process.	TRUE
The verification and validation process also includes static validation techniques.	TRUE
Testing is the only technique used in the verification and validation process. <i>☞ V&V also includes STATIC validation techniques.</i>	FALSE
Static validation techniques are NOT part of the verification and validation process. <i>☞ They ARE part of V&V.</i>	FALSE

SLIDE 4 — Program Testing Goals

Statement (from Slide)	T / F
One goal of testing is to demonstrate to the developer and the customer that the software meets its requirements.	TRUE

For custom software, there should be at least one test for every requirement in the requirements document.	TRUE
For custom software, there should be at least two tests for every requirement in the requirements document. <i>🔗 At LEAST ONE test per requirement, not two.</i>	FALSE
For generic software products, there should be tests for all of the system features, plus combinations of these features.	TRUE
For generic software products, testing only the individual features (without combinations) is sufficient. <i>🔗 Combinations of features must ALSO be tested.</i>	FALSE
Another goal of testing is to discover situations in which the behavior of the software is incorrect, undesirable or does not conform to its specification.	TRUE
Defect testing is concerned with rooting out undesirable system behavior.	TRUE
Examples of undesirable system behavior include system crashes, unwanted interactions with other systems, incorrect computations and data corruption.	TRUE
Defect testing is only concerned with system crashes. <i>🔗 It covers crashes, unwanted interactions, incorrect computations, AND data corruption.</i>	FALSE
The goal of testing is only to demonstrate that the software meets its requirements. <i>🔗 There are TWO goals: validation AND discovering defects.</i>	FALSE

SLIDE 5 — Validation and Defect Testing

Statement (from Slide)	T / F
The first goal of testing (showing software meets requirements) leads to validation testing.	TRUE
The first goal of testing leads to defect testing. <i>🔗 First goal → VALIDATION testing. Second goal → DEFECT testing.</i>	FALSE
In validation testing, you expect the system to perform correctly using test cases that reflect the system's expected use.	TRUE
The second goal of testing (discovering incorrect behavior) leads to defect testing.	TRUE
In defect testing, the test cases are designed to expose defects.	TRUE
Test cases in defect testing can be deliberately obscure.	TRUE
Test cases in defect testing must reflect how the system is normally used. <i>🔗 Defect test cases need NOT reflect normal system use — they can be deliberately obscure.</i>	FALSE
Defect test cases need not reflect how the system is normally used.	TRUE
Validation test cases reflect the system's expected use.	TRUE
Validation test cases are designed to be deliberately obscure. <i>🔗 DEFECT test cases can be obscure, not validation test cases.</i>	FALSE

SLIDE 6 — Testing Process Goals

Statement (from Slide)	T / F
------------------------	-------

The goal of validation testing is to demonstrate to the developer and the system customer that the software meets its requirements.	TRUE
A successful validation test shows that the system operates as intended.	TRUE
A successful validation test shows that the system has a defect. <i>☞ A successful DEFECT test exposes a defect. A successful VALIDATION test shows the system works as intended.</i>	FALSE
The goal of defect testing is to discover faults or defects in the software where its behavior is incorrect or not in conformance with its specification.	TRUE
A successful defect test is a test that makes the system perform incorrectly and so exposes a defect in the system.	TRUE
A successful defect test shows that the system operates correctly. <i>☞ A successful DEFECT test makes the system perform INCORRECTLY to expose a defect.</i>	FALSE
In defect testing, the behavior discovered must be incorrect or not in conformance with its specification.	TRUE

SLIDE 8 — Stages of Testing

Statement (from Slide)	T / F
There are three stages of testing: development testing, release testing, and user testing.	TRUE
There are four stages of testing. <i>☞ Only THREE stages: development, release, and user testing.</i>	FALSE
Development testing is where the system is tested during development to discover bugs and defects.	TRUE
Development testing occurs after the system is fully developed. <i>☞ Development testing occurs DURING development.</i>	FALSE
Release testing is where a separate testing team tests a complete version of the system before it is released to users.	TRUE
Release testing is performed by the same team that developed the system. <i>☞ Release testing is done by a SEPARATE testing team.</i>	FALSE
In release testing, the system is released to users before testing is complete. <i>☞ Testing happens BEFORE the system is released to users.</i>	FALSE
User testing is where users or potential users of a system test the system in their own environment.	TRUE
User testing is performed in the developer's environment. <i>☞ User testing is done in the USER'S OWN ENVIRONMENT.</i>	FALSE
Potential users (not just current users) may be involved in user testing.	TRUE

SLIDE 10 — Development Testing (3 Levels)

Statement (from Slide)	T / F
Development testing includes all testing activities carried out by the team developing the system.	TRUE
Development testing is carried out by a separate team, not the development team.	FALSE

🔗 Development testing is done BY THE TEAM DEVELOPING the system.	
Unit testing is where individual program units or object classes are tested.	TRUE
Unit testing should focus on testing the functionality of objects or methods.	TRUE
Unit testing should focus on testing component interfaces. 🔗 COMPONENT testing focuses on interfaces. Unit testing focuses on FUNCTIONALITY of objects/methods.	FALSE
Component testing is where several individual units are integrated to create composite components.	TRUE
Component testing should focus on testing component interfaces.	TRUE
Component testing should focus on testing component interactions. 🔗 SYSTEM testing focuses on interactions. Component testing focuses on INTERFACES.	FALSE
System testing is where some or all of the components in a system are integrated and the system is tested as a whole.	TRUE
System testing should focus on testing component interactions.	TRUE
System testing should focus on testing the functionality of individual methods. 🔗 UNIT testing focuses on individual methods. System testing focuses on COMPONENT INTERACTIONS.	FALSE

SLIDE 11 — Unit Testing

Statement (from Slide)	T / F
Unit testing is the process of testing individual components in isolation.	TRUE
Unit testing is a defect testing process.	TRUE
Unit testing is a validation testing process. 🔗 Unit testing is a DEFECT testing process.	FALSE
Individual functions or methods within an object can be a unit in unit testing.	TRUE
Object classes with several attributes and methods can be a unit in unit testing.	TRUE
Composite components with defined interfaces used to access their functionality can be a unit in unit testing.	TRUE
Unit testing only applies to individual functions or methods, not object classes. 🔗 Units can be individual functions, object classes, OR composite components.	FALSE
Unit testing tests components together (not in isolation). 🔗 Unit testing tests components IN ISOLATION.	FALSE

SLIDE 12 — Object Class Testing

Statement (from Slide)	T / F
Complete test coverage of a class involves testing all operations associated with an object.	TRUE
Complete test coverage of a class involves setting and interrogating all object attributes.	TRUE

Complete test coverage of a class involves exercising the object in all possible states.	TRUE
Testing only the most important operations gives complete test coverage of a class. <i>☞ Complete coverage requires testing ALL operations, ALL attributes, and ALL possible states.</i>	FALSE
Inheritance makes it more difficult to design object class tests.	TRUE
Inheritance makes it easier to design object class tests. <i>☞ Inheritance makes it MORE DIFFICULT because information to be tested is not localised.</i>	FALSE
Inheritance makes object class testing harder because the information to be tested is not localised.	TRUE

SLIDE 14 — Weather Station Testing

Statement (from Slide)	T / F
For weather station testing, test cases need to be defined for reportWeather, calibrate, test, startup and shutdown.	TRUE
A state model is used to identify sequences of state transitions to be tested.	TRUE
A state model identifies the event sequences that cause state transitions.	TRUE
An example state transition for the weather station is: Shutdown → Running → Shutdown.	TRUE
An example state transition is: Configuring → Running → Testing → Transmitting → Running.	TRUE
An example state transition is: Running → Collecting → Running → Summarizing → Transmitting → Running.	TRUE
For the weather station, test cases are only needed for the reportWeather operation. <i>☞ Test cases are needed for reportWeather, calibrate, test, startup AND shutdown.</i>	FALSE
A state model is used to identify which objects to delete from the system. <i>☞ A state model identifies sequences of STATE TRANSITIONS and event sequences to test.</i>	FALSE

SLIDE 15 — Automated Testing

Statement (from Slide)	T / F
Whenever possible, unit testing should be automated.	TRUE
In automated unit testing, tests are run and checked without manual intervention.	TRUE
Automated unit testing still requires manual checking after each test run. <i>☞ Automated testing runs and checks WITHOUT manual intervention.</i>	FALSE
In automated unit testing, you make use of a test automation framework such as JUnit.	TRUE
JUnit is an example of a test automation framework.	TRUE
Unit testing frameworks provide generic test classes that you extend to create specific test cases.	TRUE

Unit testing frameworks report success or failure, often through a GUI.	TRUE
Unit testing frameworks do not provide any reporting capability. <i>☞ They CAN report results, often through a GUI.</i>	FALSE
You create specific test cases by extending generic test classes provided by the framework.	TRUE

SLIDE 16 — Testing Strategies

Statement (from Slide)	T / F
There are two main testing strategies: partition testing and guideline-based testing.	TRUE
In partition testing, you identify groups of inputs that have common characteristics and should be processed in the same way.	TRUE
In partition testing, you should choose tests from within each group (partition).	TRUE
In partition testing, you only need to choose one single test for the entire input space. <i>☞ Tests must be chosen from WITHIN EACH partition/group.</i>	FALSE
In guideline-based testing, you use testing guidelines to choose test cases.	TRUE
Testing guidelines reflect previous experience of the kinds of errors that programmers often make.	TRUE
Guideline-based testing guidelines reflect future predicted errors. <i>☞ Guidelines reflect PREVIOUS EXPERIENCE of common programmer errors.</i>	FALSE

SLIDE 17 — Partition Testing

Statement (from Slide)	T / F
Input data and output results often fall into different classes where all members of a class are related.	TRUE
Each of these classes is an equivalence partition or domain.	TRUE
Within an equivalence partition, the program behaves in an equivalent way for each class member.	TRUE
Test cases should be chosen from each partition.	TRUE
Only one test case is needed for the entire input space, not one per partition. <i>☞ Test cases should be chosen from EACH partition.</i>	FALSE
An equivalence partition is a group where all members cause the program to behave differently. <i>☞ In an equivalence partition, the program behaves EQUIVALENTLY for all members.</i>	FALSE
In partition testing, you need to test every single value in every partition. <i>☞ You choose representative tests FROM each partition — not every value.</i>	FALSE

SLIDE 20 — Testing Guidelines (Sequences)

Statement (from Slide)	T / F
------------------------	-------

You should test software with sequences which have only a single value.	TRUE
You should use sequences of different sizes in different tests.	TRUE
Using only one fixed sequence length is sufficient for testing. <i>☞ Sequences of DIFFERENT SIZES should be used in different tests.</i>	FALSE
Tests should be derived so that the first, middle and last elements of the sequence are accessed.	TRUE
You only need to test the last element of a sequence. <i>☞ First, MIDDLE, and LAST elements must all be accessed.</i>	FALSE
You should test with sequences of zero length.	TRUE
Zero-length sequences do not need to be tested. <i>☞ Testing with zero-length sequences IS a guideline.</i>	FALSE

SLIDE 21 — General Testing Guidelines

Statement (from Slide)	T / F
You should choose inputs that force the system to generate all error messages.	TRUE
You should design inputs that cause input buffers to overflow.	TRUE
You should design inputs that prevent buffers from overflowing. <i>☞ The guideline is to CAUSE input buffers to overflow, to test boundary behavior.</i>	FALSE
You should repeat the same input or series of inputs numerous times.	TRUE
Repeating the same input is unnecessary in general testing. <i>☞ Repeating inputs IS one of the general testing guidelines.</i>	FALSE
You should force invalid outputs to be generated.	TRUE
You should force computation results to be too large or too small.	TRUE
You should only test with valid inputs that produce valid outputs. <i>☞ Guidelines explicitly require forcing invalid outputs and extreme computation results.</i>	FALSE

SLIDE 22 — Component Testing

Statement (from Slide)	T / F
Software components are often composite components made up of several interacting objects.	TRUE
You access the functionality of composite component objects through the defined component interface.	TRUE
Testing composite components should focus on showing that the component interface behaves according to its specification.	TRUE
Testing composite components should focus on testing each object's internal methods. <i>☞ Focus is on the COMPONENT INTERFACE, not internal methods — unit tests handle those.</i>	FALSE
When testing composite components, you can assume unit tests on individual objects have already been completed.	TRUE
When testing composite components, you must re-do all unit tests from scratch. <i>☞ You CAN ASSUME unit tests have been completed.</i>	FALSE

In the weather station example, the reconfiguration component includes objects that deal with each aspect of the reconfiguration.

TRUE

SLIDE 23 — Interface Testing

Statement (from Slide)	T / F
The objective of interface testing is to detect faults due to interface errors or invalid assumptions about interfaces.	TRUE
There are four types of interfaces: parameter, shared memory, procedural, and message passing.	TRUE
There are three types of interfaces. <i>☞ There are FOUR interface types.</i>	FALSE
Parameter interfaces involve data passed from one method or procedure to another.	TRUE
Shared memory interfaces involve a block of memory that is shared between procedures or functions.	TRUE
Procedural interfaces involve a sub-system that encapsulates a set of procedures to be called by other sub-systems.	TRUE
Message passing interfaces involve sub-systems requesting services from other sub-systems.	TRUE
In a parameter interface, a block of memory is shared between procedures. <i>☞ That describes SHARED MEMORY interfaces. Parameter interfaces pass DATA from one method to another.</i>	FALSE
In message passing interfaces, sub-systems share a block of memory. <i>☞ SHARED MEMORY interfaces share memory. Message passing interfaces are for requesting SERVICES.</i>	FALSE

SLIDE 25 — Interface Errors

Statement (from Slide)	T / F
There are three types of interface errors: interface misuse, interface misunderstanding, and timing errors.	TRUE
Interface misuse occurs when a calling component makes an error in its use of the interface.	TRUE
An example of interface misuse is passing parameters in the wrong order.	TRUE
Interface misuse means the called component has incorrect assumptions about the caller. <i>☞ Interface MISUNDERSTANDING (not misuse) is about incorrect assumptions. Misuse is an error in USING the interface.</i>	FALSE
Interface misunderstanding occurs when a calling component embeds assumptions about the called component's behaviour which are incorrect.	TRUE
Interface misunderstanding occurs when parameters are passed in the wrong order. <i>☞ Wrong parameter order is INTERFACE MISUSE. Misunderstanding is about incorrect ASSUMPTIONS about behavior.</i>	FALSE
Timing errors occur when the called and calling components operate at different speeds.	TRUE
Timing errors result in out-of-date information being accessed.	TRUE

Timing errors occur when parameters are in the wrong order.

🔗 Wrong parameter order = interface MISUSE. Timing errors are about speed differences causing stale data.

FALSE

SLIDE 26 — Interface Testing Guidelines

Statement (from Slide)	T / F
You should design tests so that parameters to a called procedure are at the extreme ends of their ranges.	TRUE
You should always test pointer parameters with null pointers.	TRUE
Null pointer testing is not necessary for interface testing. 🔗 Testing pointer parameters with NULL pointers is an explicit guideline.	FALSE
You should design tests which cause the component to fail.	TRUE
You should only design tests that allow the component to succeed. 🔗 Tests that CAUSE THE COMPONENT TO FAIL are explicitly required.	FALSE
Stress testing should be used in message passing systems.	TRUE
Stress testing should be used in shared memory systems. 🔗 Stress testing → MESSAGE PASSING systems. For SHARED MEMORY systems, vary the order of component activation.	FALSE
In shared memory systems, you should vary the order in which components are activated.	TRUE
In message passing systems, you should vary the order in which components are activated. 🔗 Varying activation order is for SHARED MEMORY systems, not message passing systems.	FALSE

SLIDE 27 — System Testing

Statement (from Slide)	T / F
System testing involves integrating components to create a version of the system and then testing the integrated system.	TRUE
The focus in system testing is testing the interactions between components.	TRUE
The focus in system testing is testing the functionality of individual methods. 🔗 UNIT testing focuses on individual methods. System testing focuses on COMPONENT INTERACTIONS.	FALSE
System testing checks that components are compatible.	TRUE
System testing checks that components interact correctly.	TRUE
System testing checks that components transfer the right data at the right time across their interfaces.	TRUE
System testing tests the emergent behaviour of a system.	TRUE
Emergent behavior is tested during unit testing. 🔗 Emergent behavior is tested during SYSTEM testing.	FALSE
System testing checks that components transfer data at any time regardless of timing. 🔗 Data must be transferred at the RIGHT TIME — timing is critical.	FALSE

SLIDE 28 — System and Component Testing

Statement (from Slide)	T / F
During system testing, reusable components that have been separately developed may be integrated with newly developed components.	TRUE
Off-the-shelf systems may be integrated with newly developed components during system testing.	TRUE
During system testing, only newly developed components are used. <i>☞ Reusable and off-the-shelf components may ALSO be integrated.</i>	FALSE
Components developed by different team members or sub-teams may be integrated during system testing.	TRUE
System testing is a collective rather than an individual process.	TRUE
System testing is an individual rather than a collective process. <i>☞ System testing is COLLECTIVE, not individual.</i>	FALSE
In some companies, system testing may involve a separate testing team with no involvement from designers and programmers.	TRUE
Designers and programmers are always involved in system testing. <i>☞ Some companies use a SEPARATE testing team with NO involvement from designers/programmers.</i>	FALSE

SLIDE 29 — Testing Policies

Statement (from Slide)	T / F
Exhaustive system testing is impossible.	TRUE
Exhaustive system testing is possible if enough time and resources are available. <i>☞ Exhaustive system testing is IMPOSSIBLE regardless of resources.</i>	FALSE
Testing policies can be developed to define the required system test coverage.	TRUE
All system functions that are accessed through menus should be tested.	TRUE
Only the most frequently used menu functions need to be tested. <i>☞ ALL system functions accessed through menus should be tested.</i>	FALSE
Combinations of functions accessed through the same menu must be tested.	TRUE
Individual menu functions are sufficient to test; combinations are not required. <i>☞ COMBINATIONS of functions accessed through the same menu MUST be tested.</i>	FALSE
Where user input is provided, all functions must be tested with both correct and incorrect input.	TRUE
Functions only need to be tested with correct input; incorrect input testing is optional. <i>☞ Testing with BOTH correct and incorrect input is required.</i>	FALSE

SLIDE 31 — Test-Driven Development

Statement (from Slide)	T / F
------------------------	-------

Test-driven development (TDD) is an approach to program development in which testing and code development are interleaved.	TRUE
In TDD, tests are written before code.	TRUE
In TDD, code is written before tests. <i>🔗 In TDD, TESTS are written BEFORE code — this is a critical distinction.</i>	FALSE
In TDD, passing the tests is the critical driver of development.	TRUE
In TDD, writing documentation is the critical driver of development. <i>🔗 PASSING THE TESTS is the critical driver, not documentation.</i>	FALSE
In TDD, you develop code incrementally, along with a test for that increment.	TRUE
In TDD, you develop all code first and then write all tests at the end. <i>🔗 TDD is INCREMENTAL — each small increment has a test written alongside it.</i>	FALSE
In TDD, you don't move on to the next increment until the code passes its test.	TRUE
In TDD, you can move to the next increment before the current test passes. <i>🔗 You MUST NOT move on until the current test passes.</i>	FALSE
TDD was introduced as part of agile methods such as Extreme Programming.	TRUE
TDD can only be used in agile development processes. <i>🔗 TDD can ALSO be used in plan-driven development processes.</i>	FALSE
TDD can also be used in plan-driven development processes.	TRUE
TDD was introduced as part of plan-driven development. <i>🔗 TDD was introduced as part of AGILE methods (specifically Extreme Programming).</i>	FALSE

SLIDE 33 — TDD Process Activities

Statement (from Slide)	T / F
In TDD, you start by identifying the increment of functionality that is required.	TRUE
The increment of functionality in TDD should normally be small and implementable in a few lines of code.	TRUE
The increment of functionality in TDD should be large and cover the entire system. <i>🔗 Increments should be SMALL, implementable in a FEW LINES of code.</i>	FALSE
In TDD, you write a test for the required functionality and implement it as an automated test.	TRUE
In TDD, you run the new test along with all other tests that have been implemented.	TRUE
Initially in TDD, the new test will fail because the functionality has not yet been implemented.	TRUE
Initially in TDD, the new test will pass immediately. <i>🔗 The new test WILL FAIL initially because the functionality is not yet implemented.</i>	FALSE
After implementing the functionality in TDD, you re-run the test.	TRUE
Once all tests run successfully in TDD, you move on to implementing the next chunk of functionality.	TRUE

In TDD, you move on to the next increment even if some tests are still failing.

☞ You move on *ONLY* once *ALL* tests run successfully.

FALSE

SLIDE 34 — Benefits of TDD

Statement (from Slide)	T / F
One benefit of TDD is code coverage — every code segment written has at least one associated test.	TRUE
In TDD, some code segments may have no associated tests. ☞ In TDD, <i>EVERY</i> code segment written has at <i>LEAST ONE</i> associated test.	FALSE
TDD provides regression testing — a regression test suite is developed incrementally as the program develops.	TRUE
In TDD, the regression test suite is written all at once at the end of the project. ☞ The suite is developed <i>INCREMENTALLY</i> alongside the program.	FALSE
TDD provides simplified debugging — when a test fails, it should be obvious where the problem lies.	TRUE
In TDD, when a test fails, all previously written code must be reviewed to find the error. ☞ The <i>NEWLY WRITTEN</i> code needs to be checked — the problem location is <i>OBVIOUS</i> .	FALSE
The newly written code needs to be checked and modified when a test fails in TDD.	TRUE
In TDD, the tests themselves serve as a form of system documentation.	TRUE
TDD tests describe what the code should be doing.	TRUE
TDD tests are not useful as documentation. ☞ Tests <i>ARE</i> a form of documentation that describe what the code should do.	FALSE

SLIDE 35 — Regression Testing

Statement (from Slide)	T / F
Regression testing is testing the system to check that changes have not broken previously working code.	TRUE
Regression testing checks that new features work correctly for the first time. ☞ Regression testing checks that <i>CHANGES</i> have not broken <i>PREVIOUSLY WORKING</i> code.	FALSE
In a manual testing process, regression testing is expensive.	TRUE
With automated testing, regression testing is simple and straightforward.	TRUE
With automated testing, regression testing is still expensive and time-consuming. ☞ Automated regression testing is <i>SIMPLE AND STRAIGHTFORWARD</i> .	FALSE
All tests are rerun every time a change is made to the program.	TRUE
Only the tests related to the changed code need to be rerun. ☞ <i>ALL</i> tests are rerun every time a change is made.	FALSE
Tests must run successfully before the change is committed.	TRUE

A change can be committed even if some tests are still failing.

☞ Tests **MUST** run successfully **BEFORE** the change is committed.

FALSE

SLIDE 37 — Release Testing

Statement (from Slide)	T / F
Release testing is the process of testing a particular release of a system that is intended for use outside of the development team.	TRUE
The primary goal of release testing is to convince the supplier that the system is good enough for use.	TRUE
The primary goal of release testing is to find as many bugs as possible. ☞ Primary goal = convince the SUPPLIER the system is good enough for use.	FALSE
Release testing has to show that the system delivers its specified functionality, performance and dependability.	TRUE
Release testing must show that the system does not fail during normal use.	TRUE
If a system passes release testing, it may still regularly fail during normal use. ☞ Release testing must confirm the system does NOT FAIL during normal use.	FALSE
Release testing is usually a black-box testing process.	TRUE
In release testing, tests are derived from the system specification.	TRUE
Release testing is a white-box testing process. ☞ Release testing is a BLACK-BOX testing process.	FALSE
In release testing, tests are derived from the internal code structure. ☞ Tests are derived from the SYSTEM SPECIFICATION (black-box), not internal code.	FALSE

SLIDE 38 — Release Testing vs System Testing

Statement (from Slide)	T / F
Release testing is a form of system testing.	TRUE
Release testing and system testing have no important differences. ☞ There are IMPORTANT DIFFERENCES between release and system testing.	FALSE
A separate team that has not been involved in the system development should be responsible for release testing.	TRUE
The development team is responsible for release testing. ☞ A SEPARATE TEAM (not involved in development) is responsible for release testing.	FALSE
System testing by the development team should focus on discovering bugs (defect testing).	TRUE
System testing by the development team should focus on validation testing. ☞ Development team system testing focuses on DEFECT testing (finding bugs). Release testing focuses on validation.	FALSE
The objective of release testing is to check that the system meets its requirements and is good enough for external use.	TRUE
Release testing is a form of defect testing. ☞ Release testing is a form of VALIDATION testing.	FALSE

Release testing is a form of validation testing.

TRUE

SLIDE 40 — User Testing

Statement (from Slide)	T / F
User or customer testing is a stage in the testing process in which users or customers provide input and advice on system testing.	TRUE
User testing is essential, even when comprehensive system and release testing have been carried out.	TRUE
User testing is unnecessary if comprehensive system and release testing have already been done. <i>☞ User testing is ESSENTIAL even after thorough system and release testing.</i>	FALSE
Influences from the user's working environment have a major effect on the reliability, performance, usability and robustness of a system.	TRUE
The user's working environment can be fully replicated in a testing environment. <i>☞ These influences CANNOT be replicated in a testing environment — this is why user testing is essential.</i>	FALSE
Influences from the user's working environment cannot be replicated in a testing environment.	TRUE

SLIDE 41 — Types of User Testing

Statement (from Slide)	T / F
There are three types of user testing: alpha testing, beta testing, and acceptance testing.	TRUE
There are only two types of user testing: alpha and beta. <i>☞ There are THREE types: alpha, beta, AND acceptance testing.</i>	FALSE
In alpha testing, users of the software work with the development team to test the software at the developer's site.	TRUE
Alpha testing is conducted at the user's site. <i>☞ Alpha testing is conducted at the DEVELOPER'S SITE.</i>	FALSE
In beta testing, a release of the software is made available to users to allow them to experiment and raise problems.	TRUE
In beta testing, users report problems they discover to the system developers.	TRUE
Beta testing is conducted at the developer's site. <i>☞ Beta testing is conducted at the USER'S site/environment, not the developer's.</i>	FALSE
In acceptance testing, customers test a system to decide whether it is ready to be accepted and deployed.	TRUE
Acceptance testing is primarily for generic off-the-shelf software products. <i>☞ Acceptance testing is primarily for CUSTOM systems.</i>	FALSE
Acceptance testing is primarily for custom systems.	TRUE
In acceptance testing, the developers decide if the system is ready for deployment. <i>☞ In acceptance testing, it is the CUSTOMERS who decide if the system is ready.</i>	FALSE
Alpha testing involves users and the development team working together.	TRUE

⚠ MOST LIKELY TRICK QUESTIONS — STUDY THESE EXTRA!

These are the statements examiners most commonly flip or twist. Master each one.

Statement (from Slide)	T / F
Testing can prove that a program has no errors. 🔗 CRITICAL: Testing reveals PRESENCE of errors, never ABSENCE. Cannot prove error-free.	FALSE
Testing uses real production data. 🔗 Testing ALWAYS uses ARTIFICIAL data.	FALSE
In TDD, code is written first, then tests. 🔗 In TDD, TESTS are always written BEFORE code.	FALSE
A successful defect test shows the system operates correctly. 🔗 A successful DEFECT test makes the system FAIL — that is how it exposes defects.	FALSE
Unit testing focuses on component interfaces. 🔗 COMPONENT testing focuses on interfaces. UNIT testing focuses on functionality of objects/methods.	FALSE
Release testing is done by the development team. 🔗 Release testing is done by a SEPARATE team not involved in development.	FALSE
Alpha testing is at the user's site. 🔗 ALPHA testing is at the DEVELOPER'S site. BETA testing is at the user's site.	FALSE
Acceptance testing is for generic software. 🔗 Acceptance testing is primarily for CUSTOM systems.	FALSE
In regression testing with automation, only changed code tests are re-run. 🔗 ALL tests are re-run every time any change is made.	FALSE
Inheritance makes object class testing easier. 🔗 Inheritance makes it MORE DIFFICULT because information is not localised.	FALSE
In shared memory systems, stress testing is used. 🔗 STRESS testing is for MESSAGE PASSING systems. In SHARED MEMORY, vary activation ORDER.	FALSE
Defect test cases must reflect how the system is normally used. 🔗 Defect test cases CAN BE deliberately obscure and need NOT reflect normal use.	FALSE
Exhaustive testing is possible with enough time. 🔗 Exhaustive system testing is IMPOSSIBLE — period.	FALSE
System testing is an individual process. 🔗 System testing is a COLLECTIVE process.	FALSE
Release testing is a white-box process. 🔗 Release testing is a BLACK-BOX process; tests come from the specification.	FALSE
User testing is unnecessary after thorough system testing. 🔗 User testing is ESSENTIAL even after comprehensive system and release testing.	FALSE
The user's working environment can be replicated in a test environment. 🔗 It CANNOT be replicated — that is the entire reason user testing is essential.	FALSE
In TDD, you can move to the next increment before the current test passes. 🔗 You MUST NOT move on until all tests pass.	FALSE
Testing guidelines are based on future predicted error types. 🔗 Guidelines are based on PREVIOUS EXPERIENCE of common programmer errors.	FALSE

Beta testing is conducted at the developer's site.

🔗 *Beta testing is at the USER'S site. Alpha testing is at the developer's site.*

FALSE



You now have EVERY LINE from EVERY SLIDE as True/False. Go ace that exam!